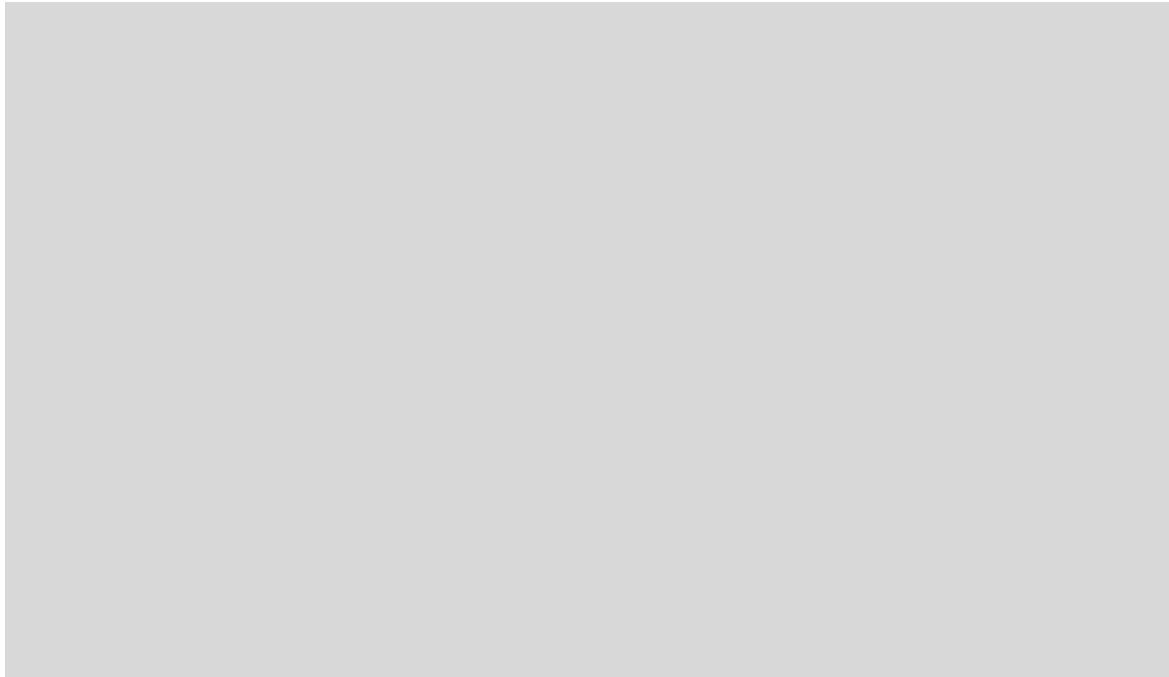


Sprites, col·lisió i animació

https://youtu.be/C_ZStbJx5uU?si=iixVyoL3mKw9_BWo



El codi per a la col·lisió (`getGlobalBounds().findIntersection()`) i el dibuixat dels sprites a `sfml` ha estat extret d'aquest tutorial de youtube. Les classes de renderer (`sprite` i `animatedSprite`) han estat realitzades en base a codi utilitzat el curs anterior a l'assignatura d'enginyeria del software on es va realitzar un projecte amb `SDL`. L'estructura de classes ha estat modificada per adaptar-se a l'estructura d'aquest projecte.

README - Justificacio d'us d'IA

Projecte: **AA3 Shooter 2D Online**

Assignatura: **Xarxes / Shooter 2D Online**

Llenguatge i entorn: **C++ / Visual Studio / SFML 3**

Aquest document explica quines parts del projecte han estat desenvolupades amb suport d'IA. La IA s'ha utilitzat com una eina d'ajuda per planificar, ordenar i generar versions inicials molt simples de certes funcionalitats. El codi final entregat no es considera una copia directa d'aquestes respostes, ja que ha estat adaptat, integrat i modificat manualment dins l'estructura real del projecte.

La justificacio esta organitzada de la seguent manera:

1. Declaracio general d'us d'IA.
2. Explicacio de com la IA ha ajudat a organitzar el projecte.
3. Llistat de funcions assistides ordenades per projecte i script.
4. Fragments de codi base generats amb IA, en versio inicial simplificada.
5. Nota final sobre autoria i responsabilitat del codi.

1. Declaracio general d'us d'IA

S'ha utilitzat IA principalment com a suport en les fases de planificacio i estructuracio del projecte. En concret, ens ha ajudat a:

- Separar responsabilitats entre client, servidor TCP/bootstrap i servidor UDP de gameplay.
- Decidir quines classes havien de gestionar cada part del flux: login, ranking, matchmaking, launcher, gameplay, paquets UDP i resultat de partida.
- Crear esquelets inicials de funcions abans d'integrar-les al projecte real.
- Revisar la logica de comunicacio TCP i UDP.
- Plantejar una estructura basica de paquets i dades compartides.
- Pensar com separar paquets continus, com el moviment, de paquets critics, com dispars o burles.
- Preparar explicacions del funcionament intern del projecte per poder justificar-lo.

La IA no s'ha utilitzat com a substitut complet del desenvolupament. Les respostes generades s'han hagut d'adaptar al projecte real, a SFML 3, a l'estructura de Visual Studio, a les classes existents, a la serialitzacio amb `sf::Packet`, a la base de dades i al comportament final del gameplay.

2. Ajuda de la IA en l'organització del projecte

La IA ha estat especialment útil per transformar una idea general de shooter online en una estructura de projecte més clara. Inicialment hi havia moltes funcionalitats barrejades: login, registre, ranking, matchmaking, launcher, gameplay i comunicació entre servidors. Amb ajuda d'IA es va plantejar separar el treball en tres blocs principals:

- **SFML_CLIENT**: aplicació gràfica del jugador. Gestiona finestres, escenes, input, render, launcher, login, lobby, ranking i gameplay local.
- **SFML_SERVER**: servidor principal TCP/bootstrap. Gestiona clients connectats, base de dades, login, registre, ranking, sales, matchmaking i comunicació inicial amb el servidor de gameplay.
- **ServerUDP**: servidor dedicat de gameplay. Gestiona la partida en temps real, estats dels jugadors, projectils, impactes, desconnexions, final de partida i comunicació UDP.

També ha ajudat a ordenar les classes segons responsabilitat:

- Les escenes del client es separen en LoginScene, LobbyScene, RankingScene i GameScene.
- La comunicació del client queda concentrada a NetworkManager.
- Les dades de protocol compartides es concentren a ProtocolData i PacketTypes.
- El servidor TCP concentra el flux de xarxa a NetworkManager i delega persistència a DatabaseConnector.
- El servidor UDP separa el socket general (GameServer) de la lògica d'una partida concreta (GameSession).
- Les tasques pesades o concurrents del servidor es recolzen en ThreadPool.

Aquesta organització no és una còpia directa d'un resultat d'IA. La IA va servir per plantejar l'estructura inicial, però el grup va adaptar els noms, fitxers, funcions, dades, comprovacions i integració final. També ha ajudat a l'organització i repartiment de tasques del equip, en aspecte de la producció. Al inici com a base, però a mesura que avançava el projecte TOTHOM, ha tocat la majoria de les diferents parts, base:

Launcher persona 1 Login/Registro/Ranking persona 2 Matchmaking persona 3
Gameplay persona 2 + 4 Thread pool persona 4 + 5

bugs y que todo funcione todos Persona 1: Iu Persona 2 + apoyo: Sergi Persona 3: Lluís
Persona 4: Carol Persona 5 + apoyo: Edgar

Persona 1 Terminar una versión offline jugable: personaje se mueve salta dispara carga
mapa .txt tiene 3 vidas cada vida tiene 5 puntos de salud respawn en el centro burla
con sonido sprites diferenciables por color

Persona 2 Montar bootstrap server: aceptar clientes por TCP enviar mapa al launcher login registro ranking básico guardar usuarios devolver resultado al cliente

Persona 3 Montar matchmaking: botón “Buscar partida amistosa” enviar petición al bootstrap cola de jugadores cuando hay 2, crear match avisar a ambos clientes pasar info al servidor dedicado

Persona 4 Montar servidor gameplay: recibir inputs simular movimiento simular disparos aplicar daño controlar vidas detectar desconexión enviar estado de partida a clientes

Persona 5 Preparar base común: PacketType.h ProtocolData.h serialización de paquetes thread pool normas de Gitflow revisión de código README bibliografía de IA/fuentes externas checklist final de entrega

FASE 1: Game offline FASE 2: Cliente conectado a bootstrap (Launcher) FASE 3: Login + menu FASE 4: Matchmaking FASE 5: Gameplay online básico FASE 6: Disparos, daño y fin de partida FASE 7: Predicción, reconciliación y pulido

3. Funcions assistides ordenades per projecte i script

En aquesta secció es llisten les parts del projecte on la IA ha ajudat més a nivell de planificació, esquelet inicial o revisió de lògica. No significa que totes aquestes funcions hagin estat generades literalment per IA; significa que formen part dels blocs on la IA va donar suport.

3.1 Projecte SFML_CLIENT

Source.cpp

Funcions relacionades:

- main

Responsabilitat:

- Inicialització del client.
- Execució del launcher abans d'entrar al joc.
- Creació de finestra i gestor d'escenes.
- Preparació del flux principal del client.

Ajuda de la IA:

- Organitzar l'ordre d'inici: launcher, connexió, login, lobby i joc.

LauncherManager.cpp

Funcions relacionades:

- LauncherManager::RunLauncher
- LauncherManager::DoMapHandshake
- LauncherManager::ReadLocalVersion
- LauncherManager::SaveMap

Responsabilitat:

- Comprovar la versio local del mapa.
- Comunicar-se amb el servidor TCP/bootstrap per verificar si el mapa esta actualitzat.
- Guardar una nova versio del mapa quan el servidor l'envia.

Ajuda de la IA:

- Plantejar el flux basic de launcher: llegir versio local, enviar-la al servidor, rebre mapa si cal i guardar-lo.

LoginScene.cpp

Funcions relacionades:

- LoginScene::OnEnter
- LoginScene::ValidateCredentials
- LoginScene::EnsureServerConnection
- LoginScene::HandleEvent
- LoginScene::Update
- LoginScene::Render
- LoginScene::OnExit

Responsabilitat:

- Mostrar la pantalla de login i registre.
- Validar que l'usuari ha introduit dades correctes abans d'enviar-les.
- Assegurar la connexio amb el servidor.
- Enviar peticions de login o registre a traves del NetworkManager.

Ajuda de la IA:

- Separar validacio local, connexio i enviament de paquets.
- Ordenar la logica de la pantalla per evitar barrejar input, xarxa i render.

LobbyScene.cpp

Funcions relacionades:

- LobbyScene::BuildUI
- LobbyScene::OnEnter
- LobbyScene::HandleEvent
- LobbyScene::AskNormalMatchmaking
- LobbyScene::AskRankedMatchmaking
- LobbyScene::CancelMatchmaking
- LobbyScene::AskCreateRoom
- LobbyScene::AskJoinRoom
- LobbyScene::Update
- LobbyScene::Render
- LobbyScene::OnExit

Responsabilitat:

- Gestionar el menu posterior al login.
- Permetre buscar partida amistosa o ranked.
- Cancel·lar matchmaking.
- Crear o unir-se a sales manuals si cal.
- Passar a gameplay quan el servidor confirma l'inici de partida.

Ajuda de la IA:

- Separar matchmaking normal i ranked.
- Plantejar una logica de cues clara al costat del client.
- Evitar que el jugador entri multiples vegades a la cua.

RankingScene.cpp

Funcions relacionades:

- RankingScene::BuildUI
- RankingScene::OnEnter
- RankingScene::HandleEvent
- RankingScene::Update
- RankingScene::Render
- RankingScene::OnExit

Responsabilitat:

- Demanar el ranking al servidor.
- Mostrar les entrades rebudes.
- Gestionar estat de carrega i missatges d'error.

Ajuda de la IA:

- Crear un flux simple de petició/resposta per consultar dades de ranking.

GameScene.cpp

Funcions relacionades:

- `GameScene::OnEnter`
- `GameScene::HandleEvent`
- `GameScene::Update`
- `GameScene::ResolveCollisions`
- `GameScene::ResolvePlayerCollision`
- `GameScene::ClampLocalPlayerToMap`
- `GameScene::SendTransform`
- `GameScene::SendTaunt`
- `GameScene::PlayTaunt`
- `GameScene::ApplyPlayerHit`
- `GameScene::Render`
- `GameScene::OnExit`
- `GameScene::HandleGameEnd`

Responsabilitat:

- Gestionar el gameplay del client.
- Llegir input del jugador.
- Actualitzar moviment local.
- Enviar transformacions al servidor UDP.
- Enviar dispars i burles.
- Rebre i aplicar estats autoritatius del servidor.
- Renderitzar jugadors, mapa, bales i HUD.
- Gestionar final de partida.

Ajuda de la IA:

- Plantejar la separació entre predicció local i estat autoritatiu del servidor.
- Organitzar enviament periòdic de transformacions.
- Diferenciar events continus de gameplay dels events crítics.

NetworkManager.cpp

Funcions relacionades:

- `NetworkManager::ConnectToServer`
- `NetworkManager::DisconnectFromServer`
- `NetworkManager::NetworkFetch`
- `NetworkManager::ReceiveData`

- NetworkManager::ClearGameNetworkState
- NetworkManager::SendUdpHelloReady
- NetworkManager::Connect
- NetworkManager::CloseConnection
- NetworkManager::SendCreateRoomRequest
- NetworkManager::SendJoinRoomRequest
- NetworkManager::SendMatchmakingRequest
- NetworkManager::SendCancelMatchmakingRequest
- NetworkManager::IsConnected
- NetworkManager::GetClientState
- NetworkManager::ProcessPacket
- NetworkManager::SendToServer
- NetworkManager::SendLoginRequest
- NetworkManager::SendRegisterRequest
- NetworkManager::SendRankingRequest
- NetworkManager::SendUdp
- NetworkManager::SendCriticalShoot
- NetworkManager::SendCriticalTaunt
- NetworkManager::ReceiveUdpData
- NetworkManager::HandleTransform
- NetworkManager::HandleShootReplicate
- NetworkManager::HandlePlayerHit
- NetworkManager::HandlePlayerTaunt
- NetworkManager::HandleEndgame
- NetworkManager::HandleCriticalAck
- NetworkManager::SendCriticalAck
- NetworkManager::UpdateCriticalPackets
- NetworkManager::StoreReceivedCriticalPacket
- NetworkManager::StorePendingCriticalPacket
- NetworkManager::CreateCriticalPacketId
- NetworkManager::HandleCreateRoomResponse
- NetworkManager::HandleJoinRoomResponse
- NetworkManager::HandleRoomStatusUpdate
- NetworkManager::HandleStartGame
- NetworkManager::HandleErrorMessage
- NetworkManager::HandleLoginResponse
- NetworkManager::HandleRegisterResponse
- NetworkManager::HandleRankingResponse

Responsabilitat:

- Centralitzar tota la comunicació TCP i UDP del client.
- Enviar peticions al servidor TCP: login, registre, ranking, creació de sala, join i matchmaking.
- Obrir i utilitzar comunicació UDP amb el servidor de gameplay.
- Gestionar paquets crítics amb ACK i reintent.
- Actualitzar l'estat compartit del client.

Ajuda de la IA:

- Separar TCP per fluxos de menú i UDP per gameplay.
- Plantejar un sistema simple de paquets crítics sobre UDP.
- Ordenar handlers segons tipus de paquet.

ClientState.cpp

Funcions relacionades:

- `ClientState::ResetRoomState`
- `ClientState::ResetAuthState`
- `ClientState::ResetRankingState`
- `ClientState::ResetAll`

Responsabilitat:

- Guardar l'estat compartit del client.
- Separar estat d'autenticació, sala, ranking i gameplay.
- Permetre reiniciar parts concretes de l'estat quan canvia d'escena.

Ajuda de la IA:

- Plantejar que el client necessitava un estat centralitzat per evitar duplicar variables entre escenes.

ProtocolData.cpp/.h i PacketTypes.h

Responsabilitat:

- Definir els tipus de paquet.
- Definir les estructures de dades compartides entre client i servidor.
- Serialitzar i deserialitzar dades amb `sf::Packet`.

Ajuda de la IA:

- Plantejar una separació clara entre tipus de paquet i dades del protocol.
- Fer més fàcil que client, servidor TCP i servidor UDP parlin el mateix idioma.

Scripts auxiliars de client

Fitxers relacionats:

- `Button.cpp`
- `InputField.cpp`
- `SceneManager.cpp`
- `TileMap.cpp`
- `Player.cpp`
- `Bullet.cpp`
- `AnimatedRenderer.cpp`
- `SpriteRenderer.cpp`
- `Entity.cpp`

Responsabilitat:

- Donar suport a UI, render, escenes, mapa, jugador, bales i entitats.

Ajuda de la IA:

- En aquests fitxers la IA s'ha utilitzat sobretot per ordenar responsabilitats i revisar fragments puntuals, no com a generació principal de tota la classe.

3.2 Projecte SFML_SERVER

`Source.cpp`

Funcions relacionades:

- `main`

Responsabilitat:

- Inicialitzar el servidor TCP/bootstrap.
- Crear les dependències principals.
- Entrar al loop principal del servidor.

Ajuda de la IA:

- Plantejar el servidor com a procés separat del client i del gameplay UDP.

`NetworkManager.cpp`

Funcions relacionades:

- `NetworkManager::Start`
- `NetworkManager::Update`
- `NetworkManager::AcceptNewClients`
- `NetworkManager::ReceiveClientData`

- NetworkManager::ProcessPacketFromPool
- NetworkManager::ProcessPacket
- NetworkManager::HandleCheckMap
- NetworkManager::HandleMapRequest
- NetworkManager::HandleRegisterRequest
- NetworkManager::HandleLoginRequest
- NetworkManager::HandleCreateRoomRequest
- NetworkManager::HandleMatchmakingRequest
- NetworkManager::HandleJoinRoomRequest
- NetworkManager::HandleEndGame
- NetworkManager::HandleRankingRequest
- NetworkManager::HandleDisconnectRequest
- NetworkManager::SendCreateRoomResponse
- NetworkManager::SendJoinRoomResponse
- NetworkManager::SendLoginResponse
- NetworkManager::SendRegisterResponse
- NetworkManager::SendErrorMessage
- NetworkManager::SendSessionToGameServer
- NetworkManager::BroadcastRoomStatus
- NetworkManager::TryStartGame
- NetworkManager::TryCreateMatchFromQueue
- NetworkManager::RemoveClientFromMatchmakingQueues
- NetworkManager::GetClientById
- NetworkManager::GetClientBySocket
- NetworkManager::RemoveDisconnectedClient
- NetworkManager::PrintConnectedClients

Responsabilitat:

- Gestionar connexions TCP dels clients.
- Processar paquets rebuts.
- Delegar tasques al ThreadPool.
- Gestionar login, registre, ranking, matchmaking i sales.
- Verificar i enviar mapa al launcher.
- Comunicar-se amb el servidor UDP per crear sessions de gameplay.
- Rebre el resultat de partida i actualitzar ranking si correspon.

Ajuda de la IA:

- Plantejar el servidor TCP com a bootstrap central.
- Ordenar handlers per tipus de paquet.
- Separar matchmaking normal i ranked.
- Separar creacio de sala de l'inici real de gameplay.

DatabaseConnector.cpp

Funcions relacionades:

- DatabaseConnector::ConnectDatabase
- DatabaseConnector::DisconnectDatabase
- DatabaseConnector::HashPassword
- DatabaseConnector::GetAllPlayers
- DatabaseConnector::LoginPlayer
- DatabaseConnector::AddPlayer
- DatabaseConnector::UpdateScore
- DatabaseConnector::GetRanking
- DatabaseConnector::UpdatePlayerScore

Responsabilitat:

- Connectar amb la base de dades.
- Registrar jugadors.
- Validar login.
- Consultar ranking.
- Actualitzar puntuacions.
- Aplicar hash a contrasenyes abans de comparar o guardar.

Ajuda de la IA:

- Separar la persistencia de dades de la logica de xarxa.
- Plantejar funcions especificques per login, registre, ranking i score.

RoomManager.cpp

Funcions relacionades:

- RoomManager::CreateRoom
- RoomManager::JoinRoom
- RoomManager::RemovePlayerFromRoom
- RoomManager::DeleteRoom
- RoomManager::GetRoom
- RoomManager::GetRoomByPlayerId
- RoomManager::RoomExists
- RoomManager::IsRoomFull
- RoomManager::PrintRooms

Responsabilitat:

- Crear i gestionar sales.

- Afegir i treure jugadors.
- Consultar si una sala existeix o esta plena.
- Relacionar jugadors amb sala.

Ajuda de la IA:

- Separar el concepte de sala del matchmaking.
- Mantenir el NetworkManager menys carregat de logica interna de rooms.

ThreadPool.cpp

Funcions relacionades:

- ThreadPool::ThreadPool
- ThreadPool::~~ThreadPool
- ThreadPool::Enqueue
- ThreadPool::Size
- ThreadPool::Worker

Responsabilitat:

- Gestionar un conjunt de threads de treball.
- Executar tasques del servidor sense bloquejar el loop principal.

Ajuda de la IA:

- Plantejar una estructura basica de thread pool per adaptar-la a la practica.
- Entendre quines tasques podien delegar-se a workers.

SHA256.cpp/.h

Funcions relacionades:

- SHA256::SHA256
- SHA256::update
- SHA256::toString
- SHA256::transform
- SHA256::pad
- SHA256::revert
- Funcions internes auxiliars com rotr, choose, majority, sig0 i sig1.

Responsabilitat:

- Generar hash de contrasenyes.
- Donar suport al login i registre amb contrasenya no guardada en text pla.

Ajuda de la IA:

- Ajudar a integrar l'us de hashing dins el flux de base de dades.

- Revisar com connectar el hash amb LoginPlayer i AddPlayer.

`ProtocolData.cpp/.h`, `PacketTypes.h`, `ConnectedClient.h` i `Room.h`

Responsabilitat:

- Definir dades compartides del servidor.
- Identificar clients connectats.
- Representar sales.
- Compartir tipus de paquets amb el client i el servidor UDP.

Ajuda de la IA:

- Plantejar una base comuna de dades per reduir desordre entre fitxers.

3.3 Projecte ServerUDP

`Source.cpp`

Funcions relacionades:

- `main`

Responsabilitat:

- Inicialitzar el servidor UDP dedicat.
- Crear el `GameServer`.
- Mantenir viu el proces de gameplay.

Ajuda de la IA:

- Separar clarament el servidor de gameplay del servidor TCP/bootstrap.

`GameServer.cpp`

Funcions relacionades:

- `GameServer::GameServer`
- `GameServer::Start`
- `GameServer::Stop`
- `GameServer::UdpReceiveLoop`
- `GameServer::UpdateLoop`
- `GameServer::HandleSessionStart`
- `GameServer::RouteUdpPacket`
- `GameServer::CleanFinishedSessions`

Responsabilitat:

- Obrir el socket UDP.

- Rebre paquets UDP.
- Crear sessions quan el servidor TCP ho demana.
- Enrutar cada paquet cap a la GameSession corresponent.
- Actualitzar sessions actives.
- Netejar sessions finalitzades.

Ajuda de la IA:

- Plantejar un router UDP simple.
- Separar recepcio general de paquets i logica especifica d'una partida.
- Diferenciar SESSION_START_REQUEST, UDP_HELLO i paquets de gameplay.

GameSession.cpp

Funcions relacionades:

- GameSession::GameSession
- GameSession::ProcessMovePacket
- GameSession::ProcessShotPacket
- GameSession::ProcessTauntPacket
- GameSession::ProcessCriticalAckPacket
- GameSession::ProcessReadyPacket
- GameSession::RegisterPlayerEndpoint
- GameSession::DisconnectPlayer
- GameSession::Update
- GameSession::IsFinished
- GameSession::GetRoomId
- GameSession::BelongsToSession
- GameSession::GetPlayerIdByAddress
- GameSession::BroadcastGameState
- GameSession::SendToPlayer
- GameSession::SendToOther
- GameSession::SendCriticalToPlayer
- GameSession::SendCriticalAck
- GameSession::UpdateCriticalPackets
- GameSession::StoreProcessedCriticalPacket
- GameSession::CreateCriticalPacketId
- GameSession::UpdateBullets
- GameSession::LoadCollisionMap
- GameSession::IsWallAt
- GameSession::SegmentHitsWall
- GameSession::PointHitsPlayer

- `GameSession::SegmentHitsPlayer`
- `GameSession::HandleHit`
- `GameSession::RespawnPlayer`
- `GameSession::PredictPositions`
- `GameSession::ClampPositionToMapBottom`
- `GameSession::CheckDisconnects`
- `GameSession::CheckHelloTimeout`
- `GameSession::SendPlayerDisconnected`
- `GameSession::FinishGame`
- `GameSession::ReportGameResult`
- `GameSession::GetState`
- `GameSession::GetIndex`
- `GameSession::GetOtherPlayerId`

Responsabilitat:

- Gestionar una partida concreta.
- Registrar endpoints UDP dels jugadors.
- Processar moviment, dispars i burles.
- Validar moviment rebut.
- Actualitzar projectils.
- Comprovar col.lisions amb parets i jugadors.
- Aplicar dany, vides i respawn.
- Enviar estat autoritatiu als clients.
- Gestionar ACK de paquets crítics.
- Detectar desconnexions.
- Finalitzar la partida i reportar resultat.

Ajuda de la IA:

- Plantejar l'estructura autoritativa de la partida.
- Dividir la logica de gameplay en funcions petites.
- Pensar un sistema de projectils real en comptes de hitscan simple.
- Afegir comprovacions basiques contra irregularitats.
- Separar moviment continu de paquets crítics.

`ThreadPool.cpp`

Funcions relacionades:

- `ThreadPool::ThreadPool`
- `ThreadPool::~~ThreadPool`
- `ThreadPool::Enqueue`

- `ThreadPool::Size`
- `ThreadPool::Worker`

Responsabilitat:

- Donar suport a execucio concurrent de tasques del servidor UDP.

Ajuda de la IA:

- Reutilitzar la idea general de thread pool en el servidor dedicat.

ProtocolData.cpp/.h i PacketTypes.h

Responsabilitat:

- Definir dades de gameplay UDP.
- Compartir tipus de paquets amb client i servidor TCP.
- Donar suport a headers, flags, ACK, transformacions, dispars, burles, hits i endgame.

Ajuda de la IA:

- Plantejar un protocol mes ordenat i ampliable.

4. Codi base generat per IA i posterior desenvolupament

Els fragments següents representen exemples de codi inicial que la IA podia generar per orientar el desenvolupament. S'han escrit com a versions simples i reduïdes del que finalment existeix al projecte. No son el codi final literal del repositori.

L'objectiu d'aquests fragments es justificar la base assistida per IA i mostrar com despres es va evolucionar cap a la implementacio final.

4.1 Launcher i comprovacio de mapa

Codi base generat amb IA:

```
bool RunLauncher()
{
    int localVersion = ReadLocalVersion();

    Packet packet;
    packet << CHECK_MAP << localVersion;
    socket.send(packet);

    Packet response;
    socket.receive(response);
```

```

bool needsUpdate;
response >> needsUpdate;

if (needsUpdate)
{
    string newMap;
    int newVersion;
    response >> newVersion >> newMap;

    SaveMap(newVersion, newMap);
}

return true;
}

```

Resultat final relacionat:

- SFML_CLIENT/LauncherManager.cpp
 - LauncherManager::RunLauncher
 - LauncherManager::DoMapHandshake
 - LauncherManager::ReadLocalVersion
 - LauncherManager::SaveMap
- SFML_SERVER/NetworkManager.cpp
 - NetworkManager::HandleCheckMap
 - NetworkManager::HandleMapRequest

Evolucio respecte al codi inicial:

- Integracio real amb sf::Packet.
- Separacio entre check de versio i peticio de mapa.
- Lectura i escriptura de fitxers locals.
- Gestio d'errors de connexio i fitxer.

4.2 Login i registre TCP

Codi base generat amb IA:

```

bool SendLoginRequest(string username, string password)
{
    if (!connected)
        return false;

    LoginRequestData data;

```

```

    data.username = username;
    data.password = password;

    Packet packet;
    packet << LOGIN_REQUEST;
    packet << data;

    return socket.send(packet) == Done;
}

void HandleLoginRequest(Client& client, Packet& packet)
{
    LoginRequestData data;
    packet >> data;

    bool ok = database.LoginPlayer(data.username, data.password);

    LoginResponseData response;
    response.success = ok;
    response.message = ok ? "Login correcto" : "Login incorrecto";

    SendLoginResponse(client, response);
}

```

Resultat final relacionat:

- SFML_CLIENT/LoginScene.cpp
 - LoginScene::ValidateCredentials
 - LoginScene::EnsureServerConnection
- SFML_CLIENT/NetworkManager.cpp
 - NetworkManager::SendLoginRequest
 - NetworkManager::SendRegisterRequest
 - NetworkManager::HandleLoginResponse
 - NetworkManager::HandleRegisterResponse
- SFML_SERVER/NetworkManager.cpp
 - NetworkManager::HandleLoginRequest
 - NetworkManager::HandleRegisterRequest
- SFML_SERVER/DatabaseConnector.cpp
 - DatabaseConnector::LoginPlayer
 - DatabaseConnector::AddPlayer
 - DatabaseConnector::HashPassword

Evolucio respecte al codi inicial:

- Validació local abans d'enviar dades.
- Separació entre escena i xarxa.
- Integració amb base de dades.
- Hash de contrasenyes.
- Respostes amb missatge i estat del jugador.

4.3 Ranking

Codi base generat amb IA:

```
void OpenRanking()
{
    staterankingLoading = true;
    stateranking.clear();

    network.SendRankingRequest(state.nickname);
}

void HandleRankingRequest(Client& client, Packet& packet)
{
    RankingRequestData request;
    packet >> request;

    RankingResponseData response;
    response.success = true;
    response.entries = database.GetRanking();

    SendRankingResponse(client, response);
}
```

Resultat final relacionat:

- SFML_CLIENT/RankingScene.cpp
 - RankingScene::OnEnter
 - RankingScene::Update
 - RankingScene::Render
- SFML_CLIENT/NetworkManager.cpp
 - NetworkManager::SendRankingRequest
 - NetworkManager::HandleRankingResponse
- SFML_SERVER/NetworkManager.cpp
 - NetworkManager::HandleRankingRequest
- SFML_SERVER/DatabaseConnector.cpp

- DatabaseConnector::GetRanking
- DatabaseConnector::UpdateScore
- DatabaseConnector::UpdatePlayerScore

Evolucio respecte al codi inicial:

- Pantalla propia de ranking.
- Estat de carrega i missatges d'error.
- Dades reals de base de dades.
- Actualitzacio del score a partir del resultat de partides ranked.

4.4 Matchmaking normal i ranked

Codi base generat amb IA:

```
void AskMatchmaking(bool ranked)
{
    if (state.isSearchingMatch)
        return;

    state.isSearchingMatch = true;
    network.SendMatchmakingRequest(ranked, state.nickname,
state.udpPort);
}

void HandleMatchmakingRequest(Client& client, bool ranked)
{
    vector<int>& queue = ranked ? rankedQueue : normalQueue;

    RemoveClientFromQueues(client.playerId);
    queue.push_back(client.playerId);

    if (queue.size() >= 2)
        CreateMatch(queue, ranked);
}
```

Resultat final relacionat:

- SFML_CLIENT/LobbyScene.cpp
 - LobbyScene::AskNormalMatchmaking
 - LobbyScene::AskRankedMatchmaking
 - LobbyScene::CancelMatchmaking
- SFML_CLIENT/NetworkManager.cpp
 - NetworkManager::SendMatchmakingRequest

- NetworkManager::SendCancelMatchmakingRequest
- SFML_SERVER/NetworkManager.cpp
 - NetworkManager::HandleMatchmakingRequest
 - NetworkManager::TryCreateMatchFromQueue
 - NetworkManager::TryStartGame
 - NetworkManager::RemoveClientFromMatchmakingQueues

Evolucio respecte al codi inicial:

- Separacio real entre cua normal i cua ranked.
- Creacio de roomId diferent segons tipus de partida.
- Evitar duplicats dins cues.
- Enviament de dades al servidor UDP quan la sala esta preparada.

4.5 Enllac entre servidor TCP i servidor UDP

Codi base generat amb IA:

```
void TryStartGame(Room& room)
{
    if (!room.IsFull())
        return;

    SessionStartData data;
    data.roomId = room.id;
    data.playerA = room.players[0];
    data.playerB = room.players[1];

    SendSessionToGameServer(data);
    NotifyPlayersGameStarted(room);
}

void HandleSessionStart(Packet& packet)
{
    SessionStartData data;
    packet >> data;

    sessions.push_back(GameSession(data));
}
```

Resultat final relacionat:

- SFML_SERVER/NetworkManager.cpp
 - NetworkManager::TryStartGame

- NetworkManager::SendSessionToGameServer
- ServerUDP/GameServer.cpp
 - GameServer::HandleSessionStart
 - GameServer::RouteUdpPacket
- ServerUDP/GameSession.cpp
 - GameSession::GameSession
 - GameSession::RegisterPlayerEndpoint
 - GameSession::ProcessReadyPacket

Evolucio respecte al codi inicial:

- El servidor TCP crea la sessio nomes quan hi ha dos jugadors.
- El servidor UDP registra endpoints reals amb UDP_HELLO.
- Els clients avisen quan estan preparats.
- La sessio comenca quan el servidor ja pot identificar els jugadors.

4.6 Moviment UDP i estat autoritatiu

Codi base generat amb IA:

```
void SendTransform()
{
    TransformData data;
    data.playerId = myId;
    data.x = player.position.x;
    data.y = player.position.y;
    data.packetId = nextPacketId++;

    Packet packet;
    packet << TRANSFORM << data;
    udpSocket.send(packet, serverIp, serverPort);
}

void ProcessMovePacket(int playerId, Packet& packet)
{
    TransformData data;
    packet >> data;

    PlayerState& state = GetState(playerId);

    if (data.packetId <= state.lastPacketId)
        return;

    state.position = { data.x, data.y };
```

```

    state.lastPacketId = data.packetId;
}

```

Resultat final relacionat:

- SFML_CLIENT/GameScene.cpp
 - GameScene::Update
 - GameScene::SendTransform
 - GameScene::ClampLocalPlayerToMap
- SFML_CLIENT/NetworkManager.cpp
 - NetworkManager::SendUdp
 - NetworkManager::ReceiveUdpData
 - NetworkManager::HandleTransform
- ServerUDP/GameSession.cpp
 - GameSession::ProcessMovePacket
 - GameSession::PredictPositions
 - GameSession::BroadcastGameState
 - GameSession::ClampPositionToMapBottom

Evolucio respecte al codi inicial:

- Capcalera UDP amb UdpPacketHeaderData.
- Control de packetId per ignorar paquets antics.
- Comprovacions de moviment massa gran.
- Predicció basica amb velocitat.
- Broadcast autoritatiu del servidor.
- Correccions per evitar caigudes sota el mapa.

4.7 Paquets critics amb ACK i reintent

Codi base generat amb IA:

```

void SendCritical(Packet packet)
{
    int id = nextCriticalId++;

    packet.header.packetId = id;
    packet.header.flags = CRITICAL;

    pendingPackets.push_back({ id, packet, 1, GetTime() });
    SendUdp(packet);
}

```

```

void UpdateCriticalPackets()

```

```

{
    for (PendingPacket& packet : pendingPackets)
    {
        if (GetTime() - packet.lastSendTime > RESEND_TIME)
        {
            SendUdp(packet.packet);
            packet.attempts++;
            packet.lastSendTime = GetTime();
        }
    }
}

void HandleAck(int packetId)
{
    RemoveFromPending(packetId);
}

```

Resultat final relacionat:

- SFML_CLIENT/NetworkManager.cpp
 - NetworkManager::SendCriticalShoot
 - NetworkManager::SendCriticalTaunt
 - NetworkManager::HandleCriticalAck
 - NetworkManager::SendCriticalAck
 - NetworkManager::UpdateCriticalPackets
 - NetworkManager::StoreReceivedCriticalPacket
 - NetworkManager::StorePendingCriticalPacket
 - NetworkManager::CreateCriticalPacketId
- ServerUDP/GameSession.cpp
 - GameSession::SendCriticalToPlayer
 - GameSession::SendCriticalAck
 - GameSession::UpdateCriticalPackets
 - GameSession::ProcessCriticalAckPacket
 - GameSession::StoreProcessedCriticalPacket
 - GameSession::CreateCriticalPacketId

Evolucio respecte al codi inicial:

- ACK en client i servidor.
- Reintents limitats.
- Historial de paquets rebuts per evitar duplicats.
- Separacio entre moviment no critic i events importants.
- Aplicacio a dispars, burles i replicacio de dispars.

4.8 Dispara, projectils i col·lisions

Codi base generat amb IA:

```
void ProcessShotPacket(int playerId, Packet& packet)
{
    ShootData data;
    packet >> data;

    Bullet bullet;
    bullet.ownerId = playerId;
    bullet.position = GetState(playerId).position;
    bullet.velocity = data.direction * BULLET_SPEED;
    bullet.active = true;

    bullets.push_back(bullet);
}

void UpdateBullets(float dt)
{
    for (Bullet& bullet : bullets)
    {
        Vector2 previous = bullet.position;
        bullet.position += bullet.velocity * dt;

        if (SegmentHitsWall(previous, bullet.position))
        {
            bullet.active = false;
            continue;
        }

        int targetId = GetOtherPlayerId(bullet.ownerId);
        if (SegmentHitsPlayer(previous, bullet.position, targetId))
        {
            bullet.active = false;
            HandleHit(targetId, bullet.ownerId);
        }
    }
}
```

Resultat final relacionat:

- SFML_CLIENT/GameScene.cpp
 - GameScene::HandleEvent
 - GameScene::ApplyPlayerHit
- SFML_CLIENT/NetworkManager.cpp
 - NetworkManager::SendCriticalShoot
 - NetworkManager::HandleShootReplicate
 - NetworkManager::HandlePlayerHit
- ServerUDP/GameSession.cpp
 - GameSession::ProcessShotPacket
 - GameSession::UpdateBullets
 - GameSession::LoadCollisionMap
 - GameSession::IsWallAt
 - GameSession::SegmentHitsWall
 - GameSession::PointHitsPlayer
 - GameSession::SegmentHitsPlayer
 - GameSession::HandleHit
 - GameSession::RespawnPlayer

Evolucio respecte al codi inicial:

- Projectils autoritatius al servidor.
- Col.lisio contra mapa carregat des de fitxer.
- Segment collision per evitar que una bala travessi parets entre frames.
- Dany, salut, vides i respawn.
- Replica visual cap als clients.

4.9 Burla sincronitzada

Codi base generat amb IA:

```
void SendTaunt()
{
    if (tauntCooldown > 0.0f)
        return;

    Packet packet;
    packet << PLAYER_TAUNT << myId;

    SendCritical(packet);
    PlayTaunt(myId);

    tauntCooldown = TAUNT_COOLDOWN;
}
```

```

void ProcessTauntPacket(int playerId, Packet& packet)
{
    SendToOther(playerId, packet);
}

```

Resultat final relacionat:

- SFML_CLIENT/GameScene.cpp
 - GameState::SendTaunt
 - GameState::PlayTaunt
- SFML_CLIENT/NetworkManager.cpp
 - NetworkManager::SendCriticalTaunt
 - NetworkManager::HandlePlayerTaunt
- ServerUDP/GameSession.cpp
 - GameSession::ProcessTauntPacket

Evolucio respecte al codi inicial:

- Cooldown de burla.
- Bloqueig temporal per evitar eco duplicat.
- Enviament com a paquet critical.
- Reproduccio sincronitzada en l'altre client.

4.10 Final de partida i ranking ranked

Codi base generat amb IA:

```

void FinishGame(int winnerId)
{
    int loserId = GetOtherPlayerId(winnerId);

    EndgameData endgame;
    endgame.winnerId = winnerId;
    endgame.loserId = loserId;

    SendToPlayer(winnerId, endgame);
    SendToPlayer(loserId, endgame);

    if (IsRankedRoom(roomId))
        ReportGameResult(winnerId, loserId);
}

void HandleEndGame(GameResultData result)
{

```

```

    if (!result.isRanked)
        return;

    database.UpdateScore(result.winnerId, WIN_POINTS);
    database.UpdateScore(result.loserId, LOSE_POINTS);
}

```

Resultat final relacionat:

- ServerUDP/GameSession.cpp
 - GameSession::CheckDisconnects
 - GameSession::CheckHelloTimeout
 - GameSession::DisconnectPlayer
 - GameSession::FinishGame
 - GameSession::ReportGameResult
- SFML_SERVER/NetworkManager.cpp
 - NetworkManager::HandleEndGame
- SFML_SERVER/DatabaseConnector.cpp
 - DatabaseConnector::UpdateScore
 - DatabaseConnector::UpdatePlayerScore

Evolucio respecte al codi inicial:

- El servidor UDP decideix el guanyador.
- El servidor TCP actualitza la base de dades.
- Les partides amistoses no modifiquen ranking.
- Les ranked modifiquen punts encara que acabin per desconnexio.

4.11 ThreadPool de servidor

Codi base generat amb IA:

```

class ThreadPool
{
public:
    ThreadPool(int count)
    {
        for (int i = 0; i < count; ++i)
            workers.emplace_back(&ThreadPool::Worker, this);
    }

    void Enqueue(function<void()> task)
    {
        lock_guard<mutex> lock(queueMutex);
    }
}

```

```

        tasks.push(task);
        condition.notify_one();
    }

private:
    void Worker()
    {
        while (running)
        {
            function<void()> task = GetNextTask();
            task();
        }
    }
};

```

Resultat final relacionat:

- SFML_SERVER/ThreadPool.cpp
 - ThreadPool::ThreadPool
 - ThreadPool::~~ThreadPool
 - ThreadPool::Enqueue
 - ThreadPool::Worker
- ServerUDP/ThreadPool.cpp
 - ThreadPool::ThreadPool
 - ThreadPool::~~ThreadPool
 - ThreadPool::Enqueue
 - ThreadPool::Worker

Evolucio respecte al codi inicial:

- Integracio amb el loop dels servidors.
- Gestio de parada segura.
- Proteccio amb mutex i condition variable.
- Execucio de tasques sense bloquejar la recepcio principal.

5. Resum de parts on la IA ha tingut mes pes

Les parts on la IA ha tingut mes influencia son:

- Arquitectura general client / servidor TCP / servidor UDP.
- Organitzacio de classes i responsabilitats.
- Esquelets inicials de login, ranking i matchmaking.
- Disseny inicial del flux TCP cap al servidor UDP.

- Idea base de servidor autoritatiu.
- Estructura de paquets UDP amb packetId.
- Sistema de paquets crítics amb ACK i reintent.
- Pseudocodi de projectils i col·lisions.
- Separació entre partides amistoses i ranked.
- Preparació d'aquesta documentació.

Les parts on s'ha hagut de fer més adaptació manual són:

- Integració amb SFML 3.
- Integració amb Visual Studio.
- Serialització real amb sf::Packet.
- Gestió concreta d'estats del client.
- Connexió amb MySQL.
- Proves locals amb diversos executables.
- Ajustos de gameplay, mapa, col·lisions i respawn.
- Correcció de bugs de connexió, ordre de paquets i sincronització.

6. Nota final sobre autoria

La IA s'ha utilitzat com una eina de suport per accelerar la planificació, generar bases inicials i revisar lògiques complexes. El resultat final no és una còpia directa d'un codi generat, sinó una implementació integrada i adaptada pel grup.

El grup es responsabilitza del codi entregat, de la seva integració amb el projecte, del funcionament final i de les decisions preses durant el desenvolupament.